Trabajo Práctico 3

Recomendador personalizado de libros

Unidad 3 Procesamiento del Lenguaje

UNR · Tecnicatura Universitaria en Inteligencia Artificial
Mismos grupos y repositorio de los TP anteriores · Google Colab
Duración: dos clases de 3 horas y trabajo en casa
Fecha de entrega: a informar

Qué vamos a construir

En el TP1 obtuvieron un catálogo de libros. En el TP2 representaron sus sinopsis y buscaron libros similares. Ahora vamos a incorporar las preferencias de una persona para que el recomendador aprenda qué libros podrían interesarle.

Antes hay un problema. El catálogo ampliado que entrega la cátedra fue contaminado: en algunas sinopsis alguien insertó enlaces, mensajes promocionales y hasta libros que no existen, para que el recomendador muestre sus sitios. Si no lo detectamos, el lector va a valorar spam y el sistema va a terminar recomendándolo. Por eso el trabajo tiene dos partes: primero auditar el catálogo y después construir el recomendador.

En el camino van a usar casi todas las herramientas de la Unidad 3: semejanza de texto, reconocimiento de entidades, clasificación, detección de idioma y resumen automático. Van a comparar dos formas de representar el texto, TF-IDF y embeddings, en dos tareas distintas. Sigan los pasos en orden y organicen el notebook con los mismos títulos.

Paso	Temas de la unidad	Cuándo
1 Cargar y revisar los libros	Limpieza (Unidad 1) · semejanza de cadenas	Clase 1
2 Auditar el catálogo	Expresiones regulares · detección de idioma · NER · clasificación · similitud	Clase 1
3 Registrar las preferencias	—	En casa, entre clases
4 Separar los libros	Entrenamiento y prueba	Clase 2
5 Entrenar dos clasificadores	Clasificación con TF-IDF y embeddings	Clase 2
6 Comprobar las predicciones	Métricas · validación cruzada	Clase 2
7 Convertir en recomendaciones	Similitud del coseno · precision@5	Clase 2
8 Recomendar del catálogo completo	Resumen extractivo · frases sustantivas y entidades	En casa

Materiales que entregamos

Archivo	Qué contiene
catalogo_tp3.jsonl	8747 libros con slug, title, authors, genres, synopsis y otros metadatos. Algunos registros están contaminados; no se indica cuáles.

Archivo	Qué contiene
muestra_etiquetada.csv	500 libros del mismo catálogo con la columna contaminado (1 o 0). En los contaminados, la columna fragmento contiene el texto que se insertó.

La muestra está enriquecida: el 20 % de sus libros está contaminado, una proporción bastante mayor que la del catálogo completo. Ténganlo en cuenta al interpretar la precisión que midan en la muestra. Además, la muestra no incluye todas las variantes de contaminación: en el catálogo hay mensajes que sus modelos nunca vieron.

Paso 1 Cargar y revisar los libros

Carguen catalogo_tp3.jsonl . Conserven slug como identificador del libro, title para mostrar su título, authors y synopsis como texto de entrada. Eliminen registros sin sinopsis y duplicados por identificador.

Normalicen los espacios. Al descargar las páginas, las palabras en cursiva quedaron separadas por saltos de línea, por ejemplo «El primer americano\n, ambiciosa obra…». Reemplacen cada secuencia de espacios en blanco por un solo espacio. No eliminen otros caracteres todavía: algunos van a ser evidencia en el paso 2.

Revisen los duplicados exactos de sinopsis antes de borrarlos. En este catálogo hay cursos y series completas que repiten la misma sinopsis en todos sus volúmenes. Para esta práctica conserven un libro por sinopsis y anoten cuántos eliminaron.

Busquen también versiones casi iguales de una obra: el mismo autor y un título muy parecido, por ejemplo con o sin subtítulo. Comparen los títulos normalizados con una medida de la unidad (Levenshtein, Jaro-Winkler o thefuzz). Para no comparar todos contra todos, comparen solo libros del mismo autor. **Atención:** la semejanza de cadenas mide la forma, no el significado. «Conversaciones con Dios I» y «Conversaciones con Dios II» se parecen mucho y son libros distintos. Definan un criterio que combine el título con la sinopsis, muestren dos pares que decidieron unir y dos que decidieron separar, y conserven una versión de cada obra.

Al terminar este paso, muestren cinco registros y escriban cuántos libros quedaron disponibles y cuántos eliminaron por cada motivo.

Paso 2 Auditar el catálogo

El catálogo contiene registros contaminados de varios tipos. Algunos tienen enlaces evidentes, otros los disimulan, otros solo agregan texto promocional y otros son libros inventados. Ningún método detecta todos. Van a construir varios detectores, medir cada uno y combinarlos.

Dividan muestra_etiquetada.csv por libro: 70 % para desarrollar los detectores y 30 % para medirlos, con random_state=42 y stratify igual a contaminado. Midan precisión y recall de la clase contaminado en esa parte de prueba. Al final encontrarán un ejemplo de código.

2.1 Expresiones regulares

Escriban una expresión regular para URLs, correos y teléfonos y midan su precisión y su recall. Revisen los falsos positivos: hay sinopsis legítimas que mencionan sitios web reales. ¿Qué tipo de contaminación detecta bien la regex y cuál no?

2.2 Normalizar antes de buscar

Algunos enlaces están disimulados: palabras en lugar de signos, letras separadas por espacios, caracteres de otro alfabeto que se ven iguales a los latinos o caracteres invisibles. Escriban una función normalizar(texto) y

vuelvan a aplicar la regex sobre el texto normalizado. Muestren dos casos que la normalización recuperó. Pista: el módulo unicodedata permite consultar el nombre y la categoría de cada carácter. Algunos caracteres invisibles aparecen también en sinopsis reales: por sí solos no son evidencia.

2.3 Detectar el idioma de cada oración

Algunos mensajes están en otro idioma dentro de una sinopsis en español. Primero detecten el idioma de la sinopsis completa con langdetect o fasttext-langdetect y observen qué pasa con los registros contaminados. Después dividan cada sinopsis en oraciones y detecten el idioma de cada una. Las oraciones cortas, las listas de nombres propios y las citas producen errores: fijen una cantidad mínima de palabras y un umbral de probabilidad (detect_langs) y justifiquen ambos valores con la muestra de desarrollo.

2.4 Reconocer entidades

Apliquen NER con spaCy (es_core_news_lg) o con GLiNER a las oraciones de los fragmentos contaminados y a una cantidad similar de oraciones limpias. Con GLiNER pueden pedir etiquetas propias, como «sitio web», «marca comercial» o «precio». Respondan: ¿qué entidades aparecen en el texto insertado que no aparecen en una sinopsis normal? ¿Sirven para detectar o solo para explicar lo que encontraron otros detectores?

2.5 Clasificar oraciones

El texto promocional sin enlaces solo se detecta aprendiendo de ejemplos. Armen un conjunto de oraciones con los libros de desarrollo: las oraciones de fragmento son positivas y las oraciones de las sinopsis limpias son negativas. Entrenen dos clasificadores, como en el paso 5: uno con TF-IDF y otro con embeddings, ambos con LogisticRegression(class_weight="balanced", max_iter=1000). Una sinopsis es sospechosa si alguna de sus oraciones supera el umbral de probabilidad que elijan.

No usen el umbral 0.5 por defecto. Los dos modelos no producen probabilidades comparables: con el mismo umbral, uno puede marcar cinco veces más libros que el otro. Elijan el umbral de cada clasificador con precision_recall_curve sobre la muestra de desarrollo, por ejemplo el que alcanza una precisión de 0.90, y comparen TF-IDF y embeddings a igual precisión.

Respondan: ¿por qué conviene clasificar oraciones y no la sinopsis completa? ¿Qué representación detecta mejor los libros contaminados de prueba? ¿En qué tipos de contaminación gana cada una? Recuerden que en el catálogo hay mensajes que no aparecen en la muestra: piensen qué representación debería generalizar mejor a frases nuevas con el mismo sentido, y cuál detecta mejor cadenas literales como una URL.

2.6 Encontrar libros copiados

Algunos libros no existen: copian la sinopsis de otro libro del catálogo y le agregan texto. Para cada libro, busquen su vecino más parecido con la similitud del coseno (NearestNeighbors con metric="cosine" sobre TF-IDF o embeddings). Revisen los pares con similitud alta y autores distintos. ¿Cuál de los dos es la copia? Recuerden que las series legítimas también comparten texto, pero en general tienen el mismo autor. Pueden usar la similitud de Jaccard sobre conjuntos de palabras para confirmar cada par.

2.7 Combinar y entregar

Combinen los detectores; por ejemplo, un libro es sospechoso si al menos uno lo marca. Presenten una tabla con la precisión y el recall de cada detector y de la combinación sobre la parte de prueba de la muestra. Marcar todo no sirve: cada libro legítimo que eliminan es un libro que nunca se va a poder recomendar.

Apliquen el sistema al catálogo completo y guarden sospechosos.csv con las columnas slug y motivo. Quiten esos libros y guarden el resultado como catalogo_limpio.csv. Los pasos siguientes usan solo el catálogo limpio.

La cátedra corregirá sospechosos.csv con una clave oculta, sin contar los libros de la muestra etiquetada, y devolverá a cada grupo su precisión, su recall y el recall por tipo de contaminación. Se publicará una tabla con los resultados de todos los grupos. La posición en esa tabla no se califica: sirve para comparar enfoques en la defensa.

Paso 3 Registrar las preferencias de un lector

Elijan a una sola persona del grupo como lector de referencia. Mezclen el catálogo limpio una sola vez con df.sample(frac=1, random_state=42) y recorran ese orden desde el principio. Esa persona leerá el título y la sinopsis de cada libro y responderá: «¿Me interesaría leer este libro?». Agreguen una columna interes.

Respuesta del lector	Valor de interes
Sí, me interesaría leerlo	1
No me interesaría leerlo	0

Si no puede decidir, conserven ese registro con interes vacío y sigan con el siguiente libro del orden. Continúen hasta reunir al menos 100 respuestas válidas y 20 de cada clase. Como siempre avanzan por el mismo orden, la selección se puede repetir aunque necesiten más de 100 libros. La misma persona debe dar todas las respuestas, antes de observar predicciones.

Hagan la valoración en casa, entre la primera y la segunda clase: lleva alrededor de una hora. Si el lector encuentra un libro contaminado que el paso 2 no detectó, déjenlo con interes vacío y agréguenlo a sospechosos.csv con motivo «revisión manual».

Guarden preferencias.csv con slug, title, synopsis e interes. Muestren cuántos 1 y cuántos 0 tienen. Un libro sin valorar tiene preferencia desconocida: no lo conviertan en un 0.

Paso 4 Separar los libros para aprender y evaluar

Trabajen solo con los registros que tienen interes igual a 0 o 1. Utilicen el 80 % para entrenamiento y el 20 % para prueba. El modelo aprenderá con el primer conjunto y será evaluado con el segundo. Con 100 valoraciones tendrán 80 libros para entrenar y 20 para probar.

Usen train_test_split con test_size=0.2, random_state=42 y stratify igual a la columna interes. Aquí random_state=42 permite repetir la misma división. La opción stratify mantiene aproximadamente la proporción de respuestas en ambos conjuntos. Guarden en preferencias.csv una columna conjunto, con entrenamiento o prueba para cada libro.

Muestren la cantidad de libros y de respuestas positivas en cada conjunto. Usen la misma división en todos los modelos. Los libros de prueba deben permanecer fuera del entrenamiento.

Paso 5 Entrenar dos clasificadores

En los dos casos, la entrada será la sinopsis y la respuesta a aprender será interes. Utilicen una regresión logística con C=1, max_iter=1000 y class_weight="balanced". Mantengan esta configuración durante todo el trabajo; esta versión del TP no incluye búsqueda de hiperparámetros.

¿Por qué class_weight="balanced"? Con pocos ejemplos positivos, una regresión logística sin este ajuste suele predecir 0 para todos los libros de prueba: acertar la clase mayoritaria es lo que más reduce el error. La opción

da más peso a los errores en la clase menos frecuente. Es parte de la configuración fija, no un ajuste para mejorar los resultados.

Modelo A con TF IDF

Primero creen un TfidfVectorizer con su configuración por defecto. Apliquen fit_transform a las sinopsis de entrenamiento. Luego usen transform para representar las sinopsis de prueba. Así, el vocabulario y los pesos se aprenden únicamente con los textos de entrenamiento.

Entrenen LogisticRegression con los vectores de entrenamiento y sus etiquetas. Después utilicen predict sobre los vectores de prueba. El resultado debe ser una lista de 0 y 1: una predicción por cada libro reservado.

Modelo B con embeddings

Reutilicen el modelo de embeddings de oraciones multilingüe del TP2; como las sinopsis están en español, el modelo tiene que ser multilingüe. Obtengan un vector para cada sinopsis de entrenamiento y de prueba, conservando el texto natural. El modelo de embeddings queda fijo: aquí entrenamos la regresión logística que recibe esos vectores.

Creen otra LogisticRegression con la misma configuración y entrénenla con los embeddings de entrenamiento y sus etiquetas. Utilicen predict sobre los embeddings de prueba.

Operación	Para qué se utiliza
fit_transform	Aprender la representación TF-IDF y transformar entrenamiento.
transform o encode	Obtener vectores de texto según la representación utilizada.
fit	Aprender a relacionar los vectores con las preferencias.
predict	Obtener la respuesta predicha 0 o 1.

Al terminar, informen el tamaño de las matrices de cada representación y conserven las predicciones de los dos modelos.

Paso 6 Comprobar las predicciones

Comparen las predicciones con las respuestas reales de los libros de prueba. Usen classification_report y una matriz de confusión. Para la clase 1, interpreten precisión, recall y F1:

Medida	Pregunta que responde
Precisión	De los libros que predijimos de interés, ¿cuántos interesaban al lector?
Recall	De los libros que interesaban al lector, ¿cuántos detectamos?
F1	¿Qué balance logramos entre precisión y recall?

Presenten una tabla con esos tres valores para cada modelo. Si no hay predicciones positivas, utilicen zero_division=0 y aclaren lo ocurrido. Con tan pocos libros, las métricas son orientativas: acompáñenlas con la cantidad de aciertos y errores.

Con 20 libros de prueba, un solo libro cambia mucho las métricas. Para ver cuánto varían, ejecuten además una validación cruzada con RepeatedStratifiedKFold (n_splits=5, n_repeats=10, random_state=42) sobre los libros

de entrenamiento, con la misma configuración, e informen la media y el desvío del F1 de la clase 1 para cada modelo. En el modelo A usen un Pipeline, para que TfidfVectorizer se ajuste dentro de cada partición. No usen la validación cruzada para elegir parámetros.

Muestren dos libros de prueba y expliquen qué pasó con cada uno. Incluyan un error si lo hubo. Por ejemplo, un falso positivo es un libro que el modelo marcó con 1, pero que el lector había valorado con 0. No cambien las etiquetas ni ajusten el modelo para mejorar los resultados de prueba.

Paso 7 Convertir las predicciones en recomendaciones

Hasta ahora predijeron una respuesta. Para recomendar necesitamos ordenar libros. Utilicen predict_proba para obtener, con cada clasificador, la probabilidad estimada de interes=1. Verifiquen en classes_ qué columna corresponde a esa clase. Ordenen de mayor a menor y tomen los primeros cinco.

Hagan una tercera lista usando la similitud del TP2. Seleccionen los libros con interes=1 de entrenamiento, promedien sus embeddings y usen ese vector como perfil del lector. Calculen la similitud del coseno entre el perfil y cada libro de prueba; ordenen de mayor a menor.

Comparen las tres listas sobre exactamente los mismos libros de prueba: TF-IDF con clasificador, embeddings con clasificador y similitud con el perfil. El perfil solo puede construirse con los positivos de entrenamiento.

Para cada lista, cuenten cuántos de sus primeros cinco libros tenían interes=1 y dividan esa cantidad por 5. Ese resultado es precision@5. Si acertaron 3 de los 5, precision@5 es 0.60. Si en prueba hay menos de cinco positivos, ninguna lista podrá alcanzar 1.

Muestren los títulos y las valoraciones reales de cada lista. Como referencia, calculen también la proporción de 1 en todo el conjunto de prueba: esa es la precisión esperada de elegir libros al azar. Respondan si los clasificadores recomendaron mejor que la similitud y fundamenten con los resultados.

Paso 8 Recomendar libros del catálogo completo

Para cerrar la práctica, usen el clasificador con embeddings del paso 5 sobre los libros del catálogo limpio que el lector todavía no valoró. Excluyan todos los libros de preferencias.csv y los casos vistos en los que hubo abstención. Usen el mismo modelo ya entrenado, sin volver a ajustarlo. Revisen que ninguna recomendación sea otra versión de un libro ya valorado.

Obtengan la probabilidad estimada de interes=1, ordenen los libros y presenten los primeros cinco con su título y sinopsis. La persona que dio las valoraciones revisará esas recomendaciones y escribirá un comentario breve: cuáles le interesan y cuáles no.

Resumir las recomendaciones

Una sinopsis completa es larga para una lista de recomendaciones. Para cada libro recomendado, generen un resumen extractivo de dos oraciones con el método de la unidad: dividan la sinopsis en oraciones, represéntenlas con TF-IDF, calculen la similitud del coseno entre ellas y conserven las dos más centrales en su orden original. Muestren un caso en el que el resumen funcionó bien y uno en el que perdió información importante.

Describir el perfil del lector

Con spaCy, extraigan las frases sustantivas (noun_chunks) y las entidades nombradas de las sinopsis de entrenamiento con interes=1 y con interes=0. Muestren las diez más frecuentes de cada grupo, usando el lema del núcleo de cada frase para agrupar variantes. El lector debe comentar si esa lista describe lo que le gusta.

Esta revisión final sirve para observar el funcionamiento sobre nuevos candidatos. Mantengan sus comentarios separados de las métricas de prueba. Una puntuación alta expresa una estimación del modelo, no una garantía de que el libro le gustará después de leerlo.

Al terminar, deben tener una función recomendar(n=5) que devuelva el identificador, el título, la puntuación y el resumen de los libros recomendados.

Qué deben entregar / en repo ya utilizado en un apartado tp3

Archivo	Qué debe incluir
TP3_grupo.ipynb	Notebook ejecutado, organizado en los ocho pasos, con tablas, resultados y recomendaciones.
grupoNN_sospechosos.csv	Los libros marcados en el paso 2, con las columnas slug y motivo.
grupoNN_preferencias.csv	Los libros valorados, sus etiquetas y la columna conjunto. Conserven también las abstenciones identificadas.
informe.pdf	Hasta un maximo de 3 páginas con las respuestas a las preguntas siguientes.

En el informe respondan estas cinco preguntas:

1. ¿Qué detectores usaron en la auditoría, qué tipo de contaminación encontró cada uno y qué costo tiene un falso positivo?

2. ¿Qué representa la etiqueta interes y quién realizó las valoraciones?

3. ¿Cómo se compararon TF-IDF y embeddings al detectar contaminación y al clasificar preferencias? ¿En qué tarea fue mayor la diferencia y por qué?

4. ¿Algún clasificador mejoró las recomendaciones por similitud?

5. ¿Qué error encontraron y qué límite tiene el experimento?

Cómo se evaluará

No se exige una métrica mínima, ni en la auditoría ni en las recomendaciones: importa que el procedimiento sea correcto y que comprendan sus resultados.

Uso de asistentes de IA

Pueden usarlos para comprender conceptos, programar y revisar errores. Agreguen al informe dos o tres líneas explicando para qué los utilizaron. Las preferencias deben ser las del lector elegido.

Guía de ayuda para la auditoría

Cargar los archivos y dividir la muestra

La muestra se divide por libro, no por oración: si dos oraciones del mismo libro quedan una en desarrollo y otra en prueba, la medición resulta optimista.

```
import re
import pandas as pd
from sklearn.model_selection import train_test_split

catalogo = pd.read_json("catalogo_tp3.jsonl", lines=True)
muestra = pd.read_csv("muestra_etiquetada.csv", keep_default_na=False)

desarrollo, prueba = train_test_split(
    muestra,
    test_size=0.3,
    random_state=42,
    stratify=muestra["contaminado"]
)
print(len(desarrollo), len(prueba))
# Resultado esperado: 350 150
```

Dividir en oraciones y medir un detector

```
from sklearn.metrics import precision_score, recall_score

def oraciones(texto):
    partes = re.split(r"(?<=[.!?])\s+|\n", texto)
    return [p.strip() for p in partes if len(p.strip()) >= 15]

def medir(nombre, marcado, datos):
    y = datos["contaminado"]
    p = precision_score(y, marcado, zero_division=0)
    r = recall_score(y, marcado)
    print(f"{nombre}: precisión {p:.2f}, recall {r:.2f}")

URL = re.compile(r"https?://|www\.", re.IGNORECASE)
medir("regex mínima", prueba["synopsis"].str.contains(URL), prueba)
```

Esta regex mínima es solo un punto de partida: tiene buena precisión y muy poco recall. Cada detector que agreguen debería mejorar el recall sin perder demasiada precisión.

Formato de sospechosos.csv

```
sospechosos = pd.DataFrame({
    "slug": ["un-slug", "otro-slug"],
    "motivo": ["regex", "idioma"]
})
sospechosos.to_csv("sospechosos.csv", index=False)
```

Guía para usar random_state

random_state fija la semilla del proceso aleatorio. La semilla es un número que permite repetir el resultado: con los mismos datos y en el mismo orden, volver a ejecutar el código produce la misma selección o división. Así podemos comparar los modelos utilizando exactamente los mismos libros.

El número 42 es una elección convencional. Podrían usar 7, 123 u otro entero; no hay una ventaja especial en 42. En este TP lo mantenemos fijo para que el experimento sea reproducible. Si cambian los datos, su orden o la semilla, el resultado puede cambiar.

Ejemplo del paso 3 Ordenar los libros a valorar

Supongan que df es el DataFrame del catálogo limpio. Las dos llamadas siguientes mezclan los libros en el mismo orden:

```
orden_1 = df.sample(frac=1, random_state=42).copy()
orden_2 = df.sample(frac=1, random_state=42).copy()

# Comprobamos que los identificadores coinciden.
print(orden_1["slug"].equals(orden_2["slug"]))
# Resultado esperado: True
```

frac=1 indica que se mezcla el catálogo completo; random_state=42 permite repetir ese orden. El lector recorre orden_1 desde el principio hasta reunir las respuestas necesarias. Si hacen falta más libros, siguen con los próximos del mismo orden.

Ejemplo del paso 4 Dividir los libros valorados

Cuando el lector haya completado preferencias.csv, pueden separar los registros con respuesta válida así:

```
import pandas as pd
from sklearn.model_selection import train_test_split

preferencias = pd.read_csv("preferencias.csv")
validas = preferencias.dropna(subset=["interes"])

entrenamiento, prueba = train_test_split(
    validas,
    test_size=0.2,
    random_state=42,
    stratify=validas["interes"]
)

print(len(entrenamiento), len(prueba))
# Con 100 respuestas válidas: 80 20
```

test_size=0.2 reserva el 20 % para prueba. random_state=42 repite qué libros quedan en cada conjunto. stratify mantiene aproximadamente la proporción de respuestas 0 y 1. Hagan esta división una vez y reutilícenla en ambos clasificadores.